MDTest User Manual

Stationarity-aware diagnostics and statistical-error assessment for molecular-dynamics time series

Software version 2.2.0.dev7

Jerry F. Wang Haobin Wang Hai Lin

Traditional user and technical manual

November 2026

Contents

First run: the simple path **3**

1 Purpose and scope **5**

2 System requirements and installation **6**
2.1 Requirements . . . 6
2.2 Recommended installation for most users . . . 6
2.3 Advanced installation (optional) . . . 6
2.4 Verification and support . . . 6

3 Input data **8**
3.1 File layout . . . 8
3.2 Missing and non-finite values . . . 8

4 Statistical background **9**
4.1 Stationarity and correlation answer different questions . . . 9
4.2 Blocking . . . 9
4.3 The four diagnostic tests . . . 9
4.3.1 Mean-trend test . . . 9
4.3.2 Dispersion-trend test . . . 10
4.3.3 Block-mean normality test . . . 10
4.3.4 Residual positive-correlation test . . . 10
4.4 Uncertainty report . . . 10

5 Algorithms and implementation for scientific users **11**
5.1 Mann–Kendall order counting . . . 11
5.2 Multi-resolution stationarity search . . . 11
5.3 FFT autocorrelation and ESS/IACT comparator . . . 12
5.4 Shapiro–Wilk, von Neumann, and confidence intervals . . . 12
5.5 Input provenance and report integrity . . . 12

6 Choosing an analysis mode **13**

7 Jupyter Workbench: step-by-step **14**
7.1 Start the Workbench . . . 14
7.2 Stage 1: choose data . . . 14
7.3 Stage 2: choose analysis . . . 14
7.4 Stage 3: run, interpret, export . . . 15

8 Python interface **17**
8.1 Auto Safe . . . 17
8.2 Auto Compare . . . 17
8.3 ESS/IACT only . . . 17
8.4 Manual . . . 18
8.5 Convenience dispatcher . . . 18

9 Command-line and configuration-file use **19**
9.1 Command-line examples . . . 19
9.2 Configuration files . . . 19

10 Understanding the output **20**
10.1 Trajectory/start plot . . . 20
10.2 Multi-resolution p-value map . . . 20
10.3 ESS scan . . . 20
10.4 Block means . . . 20
10.5 Four-test summary . . . 20

11 Interpreting common outcomes **22**

12 Worked examples **23**
12.1 Independent variance relaxation: Auto Compare . . . 23
12.2 Independent mean drift: Auto Safe . . . 23
12.3 NarK volume . . . 24

13 Troubleshooting **25**
13.1 The file loads but the observable list is wrong . . . 25
13.2 Too few blocks . . . 25
13.3 Shapiro–Wilk repeatedly fails . . . 25
13.4 Oscillatory correlations . . . 25
13.5 Export fails after the input changed . . . 25

14 Validation, reproducibility, and provenance **26**

15 License and citation **27**

A Default parameters **28**

B Package map **29**

First run: the simple path

You do not need to link the `src` directory, set environment variables, or know Python packaging to start MDTest. Extract the entire repository ZIP, then use the supplied launcher. On Windows, it creates a private environment in a short, user-owned location `%LOCALAPPDATA%\MDTest\e`, outside the extracted folder. This avoids Windows path-length errors during JupyterLab installation. On macOS/Linux, the private environment is `.venv` beside MDTest. START installs the required software and opens the Workbench in your web browser; later starts reuse the installation.

Three steps

1. **Extract the ZIP.** For Windows, right-click the downloaded ZIP and choose *Extract All*. Open the extracted MDTest folder (the folder containing `pyproject.toml`). Do not run directly from a ZIP preview.

2. **Launch MDTest.** Choose one of the following files inside the extracted folder:

   - Windows: double-click `START_MDTEST_WINDOWS.bat`.
   - macOS: double-click `START_MDTEST_MAC.command` (or use Terminal).
   - Linux: in Terminal, run `bash` followed by `START_MDTEST_LINUX.sh`.

   Leave the terminal window open while you work.

3. **Open the notebook in your browser.** The launcher normally opens JupyterLab with `MDTestWorkbench.ipynb`. If the notebook does not run automatically, select *Run* → *Run All Cells*; if asked for a kernel, choose the local Python 3 kernel. Within the Workbench, choose a data file in *Local file*, or click *Choose file* to upload one. Once a file is received, its name and byte count appear and its columns load automatically. Choose an observable and an analysis mode, then click *Run analysis*. If a file is specified by its path instead, click *Load columns* after entering the path. Use *Export* to save results.

Requirements. Python **3.11 or later** and internet access are required for the first installation. Python is not bundled. Download it from `https://www.python.org/downloads/`; on Windows select *Add Python to PATH* if offered. A private installation normally does not require administrator privileges. On subsequent starts, installation is skipped when the local environment is already ready. On Windows, moving the source folder does not move the private environment: START simply updates its editable link to the MDTest source. If JupyterLab installation fails with a Windows long-path warning, use the updated START launcher rather than enabling system-wide long-path support. A first-time package download requires internet access; scientific code and user data remain unchanged.

If the Workbench file upload does not respond. A selected file is successfully transferred only when the Workbench shows a *File received* confirmation. If that does not appear, use the *Upload Files* button in JupyterLab's left file browser to place the data file in the MDTest folder. Then click *Refresh list* in the Workbench and choose the uploaded filename under *Local file*; the Workbench loads columns automatically. Files in the repository's `data/`, `uploads/`, and `examples/data/` directories also appear in the list. Alternatively, enter the full file path and click *Load columns*. This fallback bypasses the browser widget's in-memory upload mechanism and does not change the analysis. Use *Clear* to choose the same uploaded filename again.

Input and saved-result integrity. A first CSV record that mixes numeric and nonnumeric fields is ambiguous and is rejected instead of being silently treated as column headings. Use a textual header such as `time,observable` or repair the input. In the public Python API, `drop_nan` must be a Boolean, not a string such as `'False'`. Saved analyses now include a fingerprint of the cleaned numerical time and observable arrays, in addition to the source-file checksum. If a report is reconstructed from an older saved result and the parsed rows or indexed times are incompatible, rerun the analysis rather than reusing its old start indices. The benchmark reporter fails on malformed summary JSON unless explicit partial reporting is requested by `-keep-going`.

Common error: "No module named mdtest". This occurs when a notebook is started from a different Python installation. Close that notebook and open the Workbench using the appropriate START launcher; no manual link to `src` is needed. For installation problems see `docs/INSTALL_HELP.md` and the browser-readable `START_HERE.html`.

Try it before installing. Open the saved demonstration report `examples/workbench_demo/variance_relaxation_guided.html` in any browser. This is a *static example report*, not a live analysis session.

Chapter 1

Purpose and scope

MDTest is a post-processing program for scalar observables extracted from molecular-dynamics (MD) trajectories. Its purpose is to help answer two related but distinct questions:

1. Is the retained part of the time series compatible with a stationary production regime for the chosen observable?

2. Once a production window is selected, how much does temporal correlation reduce the number of effectively independent samples and increase the uncertainty of the mean?

The software combines the Schiferl–Wallace four-test diagnostic workflow with a multi-resolution automatic stationarity search, block-based uncertainty reporting, and an optional ESS/IACT comparison in the style of Chodera/PyMBAR. It is intended as a first-line diagnostic for quantities such as energy, temperature, volume, pressure, density, structural order parameters, or other scalar time series already extracted from a trajectory.

What MDTest does not do. It does not prove global phase-space equilibration, read raw coordinate trajectories, replace physical inspection of the simulation, or make a single scalar observable representative of every slow degree of freedom. Different observables may equilibrate on different time scales.

The one-engine design

All normal interfaces call the same computational engine. The Jupyter Workbench, public Python API, command-line interface, configuration-file mode, and compatibility launcher differ in how options are supplied and results are displayed, not in the statistical formulas.

Chapter 2

System requirements and installation

2.1 Requirements

The program requires Python 3.11 or newer. Its scientific dependencies are NumPy (at least 1.24, below 3), SciPy (at least 1.10, below 2), pandas (at least 2, below 4), and Matplotlib (at least 3.7, below 4). JupyterLab and ipywidgets are needed for the browser-based Workbench. The supplied START launchers install these packages into a private environment on the first run; no separate activation is required.

2.2 Recommended installation for most users

Follow the *First run: the simple path* chapter. In particular, first *extract* the complete distribution and then start with the Windows, macOS, or Linux launcher. The launcher checks its private environment, installs the notebook dependencies when needed, and launches JupyterLab with the correct interpreter. Under Windows, the environment is stored in a short per-user location independent of the extracted ZIP path. Leave the launcher window open until you finish.

2.3 Advanced installation (optional)

Experienced Python users who prefer their own environment may install from the extracted folder containing `pyproject.toml`:

```
python -m pip install -e ".[notebook]"
python -m jupyterlab MDTestWorkbench.ipynb
```

These commands are *not required* when using the START launchers. A separate system-wide MDTest install is unnecessary, and `src/` must not be copied into the notebook folder. The editable installation is the standard supported Python packaging mechanism for the local source tree.

2.4 Verification and support

When the Workbench launches successfully, the notebook environment check reports the MDTest version and widget availability. To diagnose a failed installation, including Windows long-path errors and messages referring to old and new folders, see `docs/INSTALL_HELP.md`. Experts can inspect status without installing with `python tools/launch_mdtest.py -check` from the package

root. The source distribution also includes independent scientific regression and test scripts under `testingScripts/`.

Chapter 3

Input data

3.1 File layout

MDTest expects a text table containing a time column and one or more scalar observables. The standard convention is:

Column	Meaning	Example
0	time coordinate	fs, ps, step index
1,2,...	scalar observables	energy, volume, RMSD, etc.

The time spacing should be approximately uniform when a physical time column is used. Supported delimiter choices include comma, space, and tab. The selected observable column must be numeric.

3.2 Missing and non-finite values

MDTest first converts data cells to numeric values. To accommodate common MD tables that contain a units or metadata row after the header, up to three rows containing nonnumeric or incomplete cells are removed automatically with a warning, even when `drop_nan=False`. If more than three such rows are found, the default behavior is to stop and ask the user to inspect the file; `drop_nan=True` (or `-drop-nan`) allows those rows to be removed explicitly. Positive or negative infinity is not removed by the small metadata-row exception: if NaN/Inf remains after the first conversion step, MDTest raises an error unless `drop_nan=True`, in which case affected rows are dropped with a warning. For reproducibility, inspect every warning and document any discarded rows that could have physical meaning.

Chapter 4

Statistical background

4.1 Stationarity and correlation answer different questions

For a retained stationary scalar process with variance σ^2 and normalized autocorrelation $C(k)$, the statistical inefficiency is

$$g = 1 + 2\sum_{k=1}^{\infty} C(k), \tag{4.1}$$

and the effective sample size is approximately

$$N_{\text{eff}} = \frac{N}{g}. \tag{4.2}$$

This corrects sampling uncertainty for correlation. It does not, by itself, test whether the retained distribution is time-independent. A time series can have an approximately stationary mean while its variance is still relaxing; conversely, a stationary series can be strongly correlated.

4.2 Blocking

For candidate start index i_s, block length m, and n complete blocks, MDTest forms nonoverlapping blocks and block means

$$Y_k = \frac{1}{m}\sum_{j=1}^{m} X_{i_s+(k-1)m+j}, \tag{4.3}$$

with within-block variance

$$V_k = \frac{1}{m-1}\sum_{j=1}^{m}\left(X_{i_s+(k-1)m+j} - Y_k\right)^2. \tag{4.4}$$

If a physical time coordinate t_i is present, the start time is $t_s = t_{i_s}$.

4.3 The four diagnostic tests

4.3.1 Mean-trend test

A two-sided Mann–Kendall test is applied to the sequence of block means. Failure indicates evidence of residual monotone drift in the retained mean. The usual response is to move the production start later or inspect whether the observable is undergoing a physical transition.

4.3.2 Dispersion-trend test

The same Mann–Kendall logic is applied to the within-block variances V_k. This detects monotone changes in fluctuation magnitude even when the mean is stable. It is especially relevant for fluctuation-derived quantities such as heat capacities, compressibilities, or dielectric response functions.

4.3.3 Block-mean normality test

The Shapiro–Wilk test is applied to the *block means*, not to the raw observable. A raw non-Gaussian observable can still yield approximately normal block means when blocks are sufficiently long and dependence is suitably weak. Persistent rejection after increasing m warns that the normal-theory von Neumann interpretation and Student-t confidence interval are not fully supported.

4.3.4 Residual positive-correlation test

A one-tailed von Neumann ratio test is applied to adjacent block means. Positive persistence makes successive block means more similar and tends to reduce the ratio below its independent-sample value. MDTest uses this as a local residual-correlation diagnostic after the normality guard has passed.

4.4 Uncertainty report

When all four checks support the chosen reporting scale, MDTest reports the block-mean average, its standard error, and a two-sided Student-t confidence interval. A passing result means only that the chosen observable/window/scale does not provide sufficient evidence against the tested assumptions; it is not a proof of complete equilibration.

Chapter 5

Algorithms and implementation for scientific users

This chapter gives enough implementation detail to understand what the program computes without assuming a computer-science background.

5.1 Mann–Kendall order counting

The Mann–Kendall statistic involves comparing every earlier block value with every later block value. A direct implementation uses about $n(n-1)/2$ comparisons and therefore scales as $\mathcal{O}(n^2)$. That is easy to understand but unnecessarily expensive inside an automatic scan over many candidate starts and block lengths.

MDTest first replaces the distinct finite values by their sorted integer ranks (*rank compression*). It then processes the ranks from left to right while maintaining cumulative counts of previously seen ranks. This lets the program obtain the number of previous values that are smaller, equal, or larger without rescanning the entire history.

Computer-science term. The cumulative-count data structure is a **Fenwick tree**, also called a **binary indexed tree** [5]. Each update and cumulative-count query costs $\mathcal{O}(\log u)$, where u is the number of distinct finite values, so the complete order count costs $\mathcal{O}(n \log u)$. The statistical Mann–Kendall definition is unchanged; only the way the pair count is computed is accelerated.

A closely related textbook idea is *inversion counting* during merge sort: while two sorted halves are merged, one can count how many elements are out of order "for free" during the merge [6]. That is a useful conceptual analogy, but the frozen MDTest implementation uses a Fenwick tree rather than merge-sort inversion counting.

5.2 Multi-resolution stationarity search

The automatic stationarity search avoids selecting a start from one arbitrary smoothing level.

1. Choose a candidate-start grid and a diagnostic block-size grid $\mathcal{M}_{\mathrm{tr}}$.
2. Reject a candidate start if any diagnostic block size would leave fewer than the required minimum number of complete blocks.
3. At each admissible start and diagnostic block size, compute both trend-test p-values.

4. Select the earliest start that passes both trend screens at every diagnostic block size.

5. With that start fixed, search a separate reporting block-size grid for the smallest scale at which the complete four-test protocol passes while retaining enough blocks. The mean- and dispersion-trend tests are therefore rechecked at each candidate reporting block size together with Shapiro–Wilk and von Neumann.

The search stops at the earliest candidate start that passes all configured diagnostic block sizes. The program therefore retains the *evaluated* p-value grid from the beginning of the scan through that selected candidate; later admissible starts are not evaluated or stored. This retained grid lets the user inspect how the approach to the selected start depends on coarse graining.

5.3 FFT autocorrelation and ESS/IACT comparator

For a retained suffix, MDTest estimates the autocorrelation sequence with a fast Fourier transform (FFT), which evaluates the correlation convolution in approximately $\mathcal{O}(N \log N)$ rather than $\mathcal{O}(N^2)$. The finite-lag sum follows the PyMBAR-style logic used in the frozen engine: positive/negative autocorrelation terms are accumulated with the finite-length weight $(1 - k/N)$ and the sum is truncated at the first non-positive correlation after the requested minimum lag. The returned statistical inefficiency is constrained to $g \geq 1$.

The ESS start scan evaluates candidate truncation points and chooses the one maximizing retained effective samples, $(N - i_s)/g(i_s)$. This is an uncertainty-efficiency criterion and is displayed separately from the stationarity start. In `auto-compare`, MDTest also reports the conservative combined start $\max(i_s^{\mathrm{SW}}, i_s^{\mathrm{ESS}})$.

5.4 Shapiro–Wilk, von Neumann, and confidence intervals

MDTest delegates the Shapiro–Wilk distribution calculation to SciPy. The von Neumann statistic and Student-t interval are computed directly from the retained block means using the formulas described in the associated manuscript. The software does not silently substitute a different statistical test when a diagnostic fails; instead it reports the failure and suggests analysis adjustments.

5.5 Input provenance and report integrity

Structured results record the input SHA-256 hash and software/dependency versions. Raw-data-dependent plots verify the current source against the analyzed hash before export. This prevents a report from combining old statistics with a file that was modified after analysis.

Chapter 6

Choosing an analysis mode

Mode	Use when	Main output
Manual	You already chose t_s, m, and n	Four-test diagnostic and uncertainty report
Auto Safe	You want MDTest stationarity screening only	i_s^{SW} plus reporting block length
ESS/IACT only	You want the correlation/ESS comparator separately	i_s^{ESS}, g, N_{eff}
Auto Compare	You want both starts side by side	SW start, ESS start, conservative combined start
Sensitivity	Legacy API/CLI diagnostic of initial-block sensitivity; not a Workbench mode	Table versus m_{init}

For new routine analyses, the Workbench exposes the first four modes directly. The legacy Sensitivity diagnostic remains available through the Python API and command line but is not a Workbench mode. Substantially different SW and ESS starts should be interpreted physically rather than treated as a software error.

Chapter 7

Jupyter Workbench: step-by-step

7.1 Start the Workbench

Launch the Workbench using the corresponding START file described in the first chapter. It starts JupyterLab with the private Python environment and opens the notebook without additional installation commands. In JupyterLab, open `MDTestWorkbench.ipynb` and execute the notebook cells. Advanced users with an already-configured Python/Jupyter environment may alternatively launch it from an existing notebook:

```
from mdtest.workbench import launch
wb = launch()
```

7.2 Stage 1: choose data

1. Choose a file in *Local file* or use *Choose file* to upload it. Columns load automatically on successful selection. For a typed path, click *Load columns*.
2. Select delimiter and missing-data policy.
3. Click **Load columns**.
4. Inspect the detected columns and choose the observable.
5. Confirm that the detected columns and selected observable correspond to the intended physical quantity.

The Workbench binds the loaded column metadata to the exact source content and parsing options. If the source changes, it requires columns to be reloaded.

7.3 Stage 2: choose analysis

Select Manual, Auto Safe, ESS/IACT only, or Auto Compare. Most users should begin with defaults. Advanced settings are available in the accordion for significance level, diagnostic/reporting block grids, candidate-start stride, ESS scan stride, and lag limits.

7.4 Stage 3: run, interpret, export

Click **Run analysis**. The integrated dashboard reports the selected starts, reporting block length, statistical inefficiency/ESS when applicable, final statistical estimates, guided interpretation, diagnostic figures, and reproducibility metadata in one result surface. Expand the guided or technical explanation for diagnostic meanings and references. Export creates structured JSON, text, figures, and a self-contained HTML report. If an inference-controlling setting is changed after a completed run, the old result is invalidated for export until the analysis is rerun.

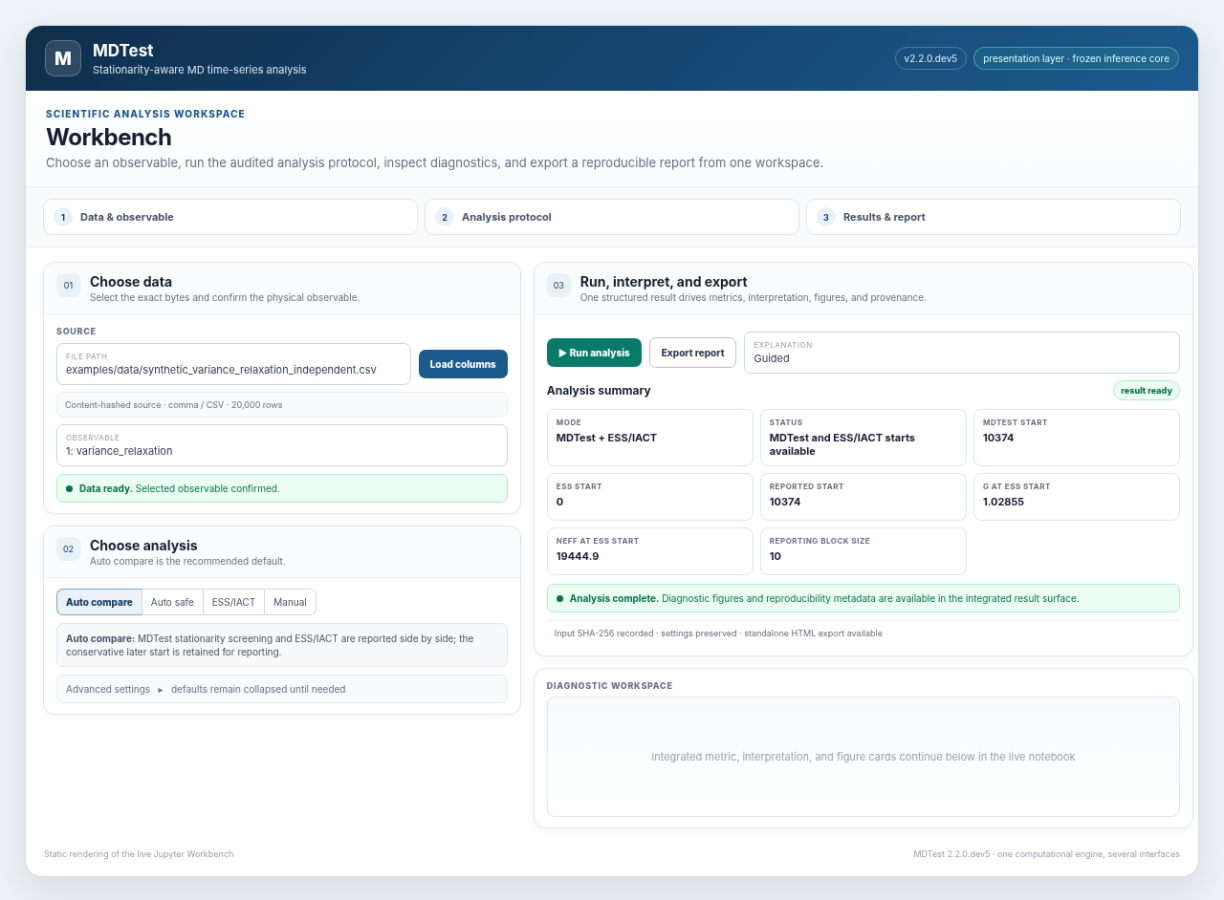


Figure 7.1: Static rendering of the MDTest Workbench layout. The distributed notebook provides the live interactive controls; the interface separates data selection, analysis choices, and the result/-export stage.

Chapter 8

Python interface

8.1 Auto Safe

```
import mdtest
r = mdtest.auto_safe(
    "trajectory.csv",
    col=1,
    m_trend_grid=[2,5,10,20,40],
    min_blocks=24,
)
print(r.selected_ts_sw)
print(r.selected_m_corr)
```

By default the Python API returns objects and does not write files. Use `write_outputs=True` and `make_plots=True` when file output is desired.

8.2 Auto Compare

```
r = mdtest.auto_compare(
    "trajectory.csv", col=1,
    write_outputs=True,
    make_plots=True,
    outdir="results/run1",
)
print(r.auto_safe.selected_ts_sw)
print(r.ess.selected_ts_ess)
print(r.selected_ts_star)
```

8.3 ESS/IACT only

```
e = mdtest.ess("trajectory.csv", col=1)
print(e.selected_g, e.selected_neff)
```

8.4 Manual

```
r = mdtest.manual("trajectory.csv", col=1,
                  ts=600000, m=30, n="max")
```

8.5 Convenience dispatcher

```
r = mdtest.analyze("trajectory.csv", mode="auto-safe", col=1)
```

Chapter 9

Command-line and configuration-file use

9.1 Command-line examples

```
mdtest manual --file trajectory.csv --col 1 --ts 600000 --m 30 --n max

mdtest auto-safe --file trajectory.csv --col 1 \
  --m-trend-grid 2,5,10,20,40 --min-blocks 24

mdtest auto-compare --file trajectory.csv --col 1 \
  --m-trend-grid 2,5,10,20,40
```

Use `mdtest <mode> -help` for mode-specific options.

9.2 Configuration files

Configuration-file mode is intended for recorded, repeatable choices. The first line must be `MD Test Format`; subsequent lines use key-value entries described in `docs/CONFIG_REFERENCE.md`. Unknown keys are rejected rather than silently ignored.

Chapter 10

Understanding the output

10.1 Trajectory/start plot

This plot overlays the raw scalar time series with the MDTest stationarity start, ESS start, and final combined start when available. Use it to verify that selected starts make physical sense in the context of visible transients or state changes.

10.2 Multi-resolution p-value map

The two panels show mean-trend and dispersion-trend p-values as functions of candidate start and diagnostic block size. The selected start must satisfy the configured trend criteria across the admissible diagnostic grid. A broad region of passing values is usually more reassuring than an isolated borderline transition.

10.3 ESS scan

The ESS scan shows g and/or N_{eff} as the candidate start changes. A large g indicates strong stationary correlation; an early ESS start does not guarantee that mean or variance transients are absent.

10.4 Block means

Inspect the retained block means for slow drift, isolated transitions, oscillatory structure, or too few blocks. The reported horizontal mean line is a statistical summary, not a substitute for the diagnostic pattern.

10.5 Four-test summary

The final panel reports the trend, normality, and residual-correlation checks at the reporting scale. "Pass" means no statistically significant violation was detected at the configured significance level; it does not prove the assumption. Numerical mean, standard-error, or confidence-interval fields can still be computed in some failing cases, but a failed diagnostic means the usual parametric interpretation is not fully supported and should be treated as a warning rather than as a validated production summary.

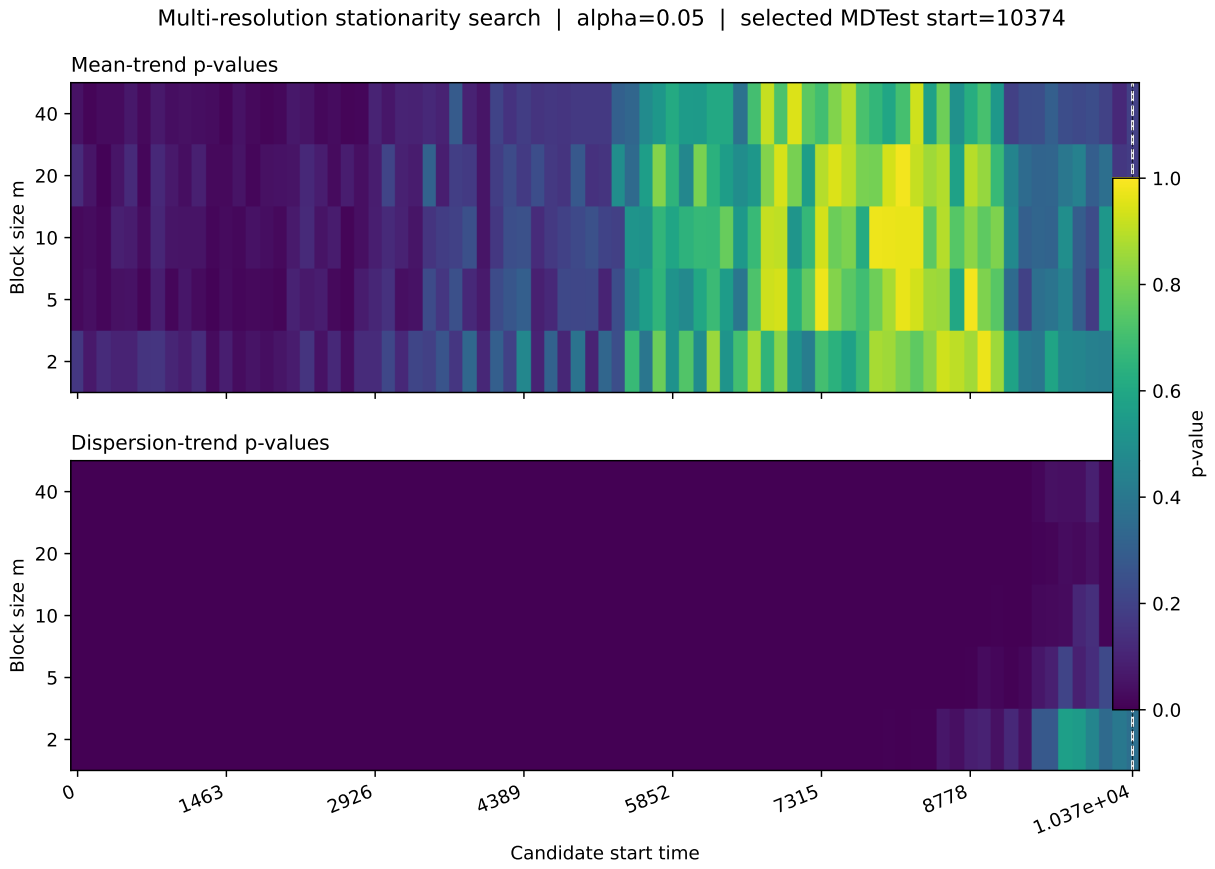


Figure 10.1: Example multi-resolution stationarity map from the released variance-relaxation benchmark.

Chapter 11

Interpreting common outcomes

Observation	Interpretation	Typical action
Mean trend fails	retained average is still changing monotonically	move start later; inspect transition
Dispersion trend fails	fluctuation magnitude changes with time	move start later; inspect variance/physical transition
Shapiro fails	block means do not support normal-theory reporting at this scale	increase m, collect more data, bootstrap/nonparametric analysis
Von Neumann fails	adjacent block means retain positive persistence	increase m or collect more data
SW start much later than ESS start	stationarity screen sees a transient not reflected in ESS efficiency	inspect mean/dispersion maps and trajectory
ESS start much later than SW start	retained data may be stationary but strongly correlated	inspect autocorrelation/ESS and use later start if following combined mode
No reporting m passes	insufficient length, persistent nonnormality, or strong correlation	longer simulation or different uncertainty method

Chapter 12

Worked examples

12.1 Independent variance relaxation: Auto Compare

This released benchmark is independent in time but has a changing fluctuation scale. It is therefore a useful demonstration of why stationarity assessment and correlation correction answer different questions. From the root of the unpacked source distribution, run:

```
import mdtest
r = mdtest.auto_compare(
    "examples/data/synthetic_variance_relaxation_independent.csv",
    col=1,
    m_trend_grid=[2, 5, 10, 20, 40],
    m_corr_grid=[10, 20, 40, 80, 160, 320],
)
print(r.auto_safe.selected_start_index_sw)
print(r.ess.selected_start_index_ess)
print(r.selected_start_index_star)
print(r.ess.selected_g)
print(r.auto_safe.selected_m_corr)
```

For the released file and frozen algorithm this prints, apart from ordinary floating-point formatting,

```
10374
0
10374
1.0285490983959442
10
```

The ESS comparator keeps the complete trajectory because there is essentially no temporal correlation in the mean, whereas the dispersion-trend screen detects the relaxing fluctuation scale and selects the later stationarity start.

12.2 Independent mean drift: Auto Safe

The companion mean-drift benchmark is also independent in time, but its mean relaxes before production. Run:

```
import mdtest
r = mdtest.auto_safe(
    "examples/data/synthetic_mean_drift_independent.csv",
```

```
    col=1,
    m_trend_grid=[2, 5, 10, 20, 40],
    m_corr_grid=[10, 20, 40, 80, 160, 320],
)
print(r.selected_start_index_sw)
print(r.selected_m_corr)
print(r.manual_result.mean)
```

The protected manuscript-scale result is

```
9709
10
-0.001293179522305782
```

These examples are regression-protected values for the released benchmark files; they are not general thresholds for other trajectories.

12.3 NarK volume

The NarK example illustrates a real correlated MD observable. The multi-resolution search reduces dependence on one initial block length; the ESS comparator reports substantial autocorrelation and may choose a later start. Use the released configuration/example files to reproduce the manuscript tables. Because the public source distribution contains the example data and configurations whereas an installed wheel need not contain the repository-level example tree, run these reproduction examples from an unpacked source distribution.

Chapter 13

Troubleshooting

13.1 The file loads but the observable list is wrong

Check delimiter, header/unit rows, and whether the selected source is the path or upload you intended. Reload columns after changing source or parsing controls. A file whose name ends in `.csv` is read as comma-delimited by the frozen loader; use a suitable text extension such as `.txt` or `.dat` for whitespace-delimited data, or convert the file to a conventional comma-separated CSV.

13.2 Too few blocks

A candidate start/block size is feasible only when enough complete blocks remain. Reduce m, move the start earlier only if scientifically justified, or collect a longer trajectory.

13.3 Shapiro–Wilk repeatedly fails

Remember that the test is on block means. Try larger blocks while keeping enough block samples. For strongly multimodal or long-memory observables, a Gaussian/Student-t summary may simply be inappropriate.

13.4 Oscillatory correlations

The default von Neumann test is one-sided for positive adjacent-block persistence. Strong underdamped oscillation can make both block-level and IACT diagnostics sensitive to scale/truncation. Inspect the raw/autocorrelation pattern and consider larger blocks or a model-based estimator.

13.5 Export fails after the input changed

This is deliberate provenance protection. Restore the analyzed file or rerun MDTest on the current file before exporting raw-data-dependent figures.

Chapter 14

Validation, reproducibility, and provenance

The scientific engine is protected by regression tests against manuscript-scale benchmark values. The source distribution includes unit/interface tests, CLI smoke tests, frozen-engine hash checks, and benchmark regeneration scripts. Structured results carry the inference-controlling settings and environment/input provenance. For publication or collaboration, archive the JSON result together with the exact input file and software version.

Chapter 15

License and citation

MDTest is distributed under the MIT License. The license permits use, modification, publication, and redistribution of the software and associated documentation, provided the copyright and permission notice are retained in copies or substantial portions. The software is provided without warranty; see the distributed `LICENSE` file for the controlling text.

The detailed manual is therefore appropriate to distribute inside the software package. The journal manuscript should summarize the program and point readers to the distributed manual rather than reproducing the manual itself.

For scientific use, cite the associated MDTest manuscript and the primary methods relevant to the analysis.

Appendix A

Default parameters

Quantity	Default	Meaning
α	0.05	diagnostic significance level
$n_{\min}$	24	minimum retained blocks for MDTest block diagnostics
Trend grid	2,5,10,20,40	diagnostic block sizes in automatic stationarity search
Reporting grid	5,10,20,30,40,60,80,120,160,240,320,480	candidate reporting block sizes
ESS minimum lag	3	earliest lag after which a non-positive ACF can terminate integration

Defaults are starting points, not universal physical constants. Users should inspect whether block sizes are sensible relative to the dominant time scales of the observable.

Appendix B

Package map

src/mdtest/api.py public Python entry points.

src/mdtest/analysis.py automatic/manual orchestration using the frozen statistical primitives.

src/mdtest/diagnostics.py block construction and four-test primitives.

src/mdtest/autocorrelation.py FFT autocorrelation, statistical inefficiency, ESS-start scan.

src/mdtest/results.py structured result/settings/provenance dataclasses.

src/mdtest/workbench.py Jupyter Workbench controller and widgets; no duplicate statistical engine.

src/mdtest/visualization.py Workbench diagnostic figures.

src/mdtest/presentation.py guided/technical text and HTML/report export.

docs/ architecture, API, configuration, Workbench, scientific-freeze, and audit documentation.

examples/ released data, configurations, and Workbench demonstration.

testingScripts/ scientific regression and packaging checks.

Bibliography

[1] S.K. Schiferl and D.C. Wallace, Statistical errors in molecular dynamics averages, J. Chem. Phys. 83 (1985) 5203–5209.

[2] H.B. Mann, Nonparametric tests against trend, Econometrica 13 (1945) 245–259.

[3] M.G. Kendall, Rank Correlation Methods, 4th ed., Charles Griffin, London, 1975.

[4] S.S. Shapiro and M.B. Wilk, An analysis of variance test for normality (complete samples), Biometrika 52 (1965) 591–611.

[5] P.M. Fenwick, A new data structure for cumulative frequency tables, Software: Practice and Experience 24 (1994) 327–336. doi:10.1002/spe.4380240306.

[6] J. Kleinberg and E. Tardos, Algorithm Design, Addison-Wesley, 2006 (merge-sort inversion counting as a related textbook technique).

[7] H. Flyvbjerg and H.G. Petersen, Error estimates on averages of correlated data, J. Chem. Phys. 91 (1989) 461–466.

[8] J.D. Chodera, A simple method for automated equilibration detection in molecular simulations, J. Chem. Theory Comput. 12 (2016) 1799–1805.